软件演示 | DEMO WORKFLOW

用 OpenBioWorkstation 复现 WSMV/TriMV 分析

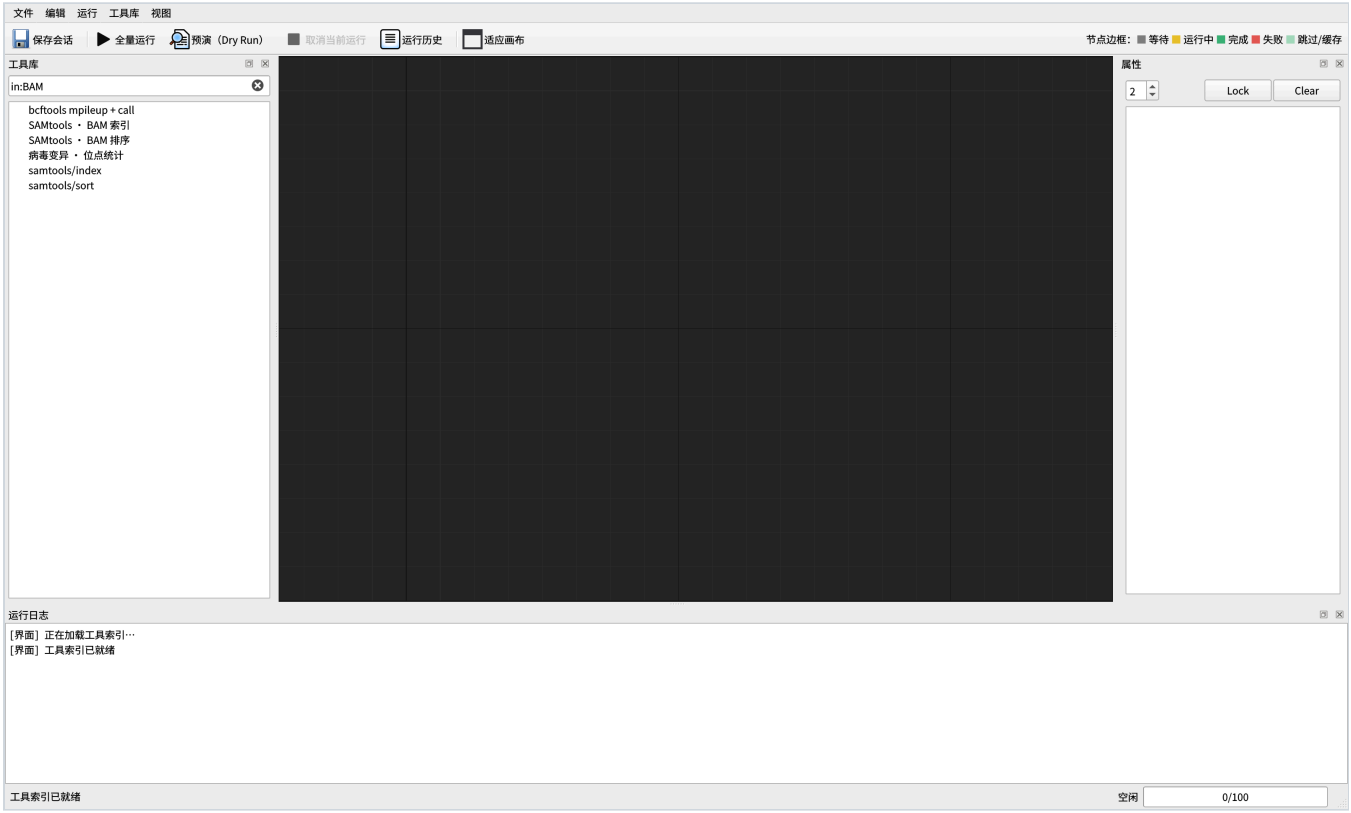
OpenBioWorkstation 课题组（自动化分析）

本演示按实际操作顺序排列 11 张真实界面截图与 1 张同一截图的局部放大（offscreen 渲染，1600×960 逻辑像素，2× 设备像素）。流程覆盖：空工作区与工具库过滤、MCP 自动封装、完整画布与局部特写、共享节点接线、端口类型拦截、运行中、运行完成、输入/输出预览、单步重跑缓存与运行历史。

> **演示环境：**工作区为课题目录 studies/wsmv_coinfection/；Conda 环境 ~/.cache/biows/common-tools（fastp 1.4.0、BWA 0.7.19-r1273、SAMtools 1.25、bcftools 1.25、seqkit）；运行使用 --cores 4。截图中的样本名、节点名与状态均为真实运行结果。

步骤 1　空工作区与工具库过滤

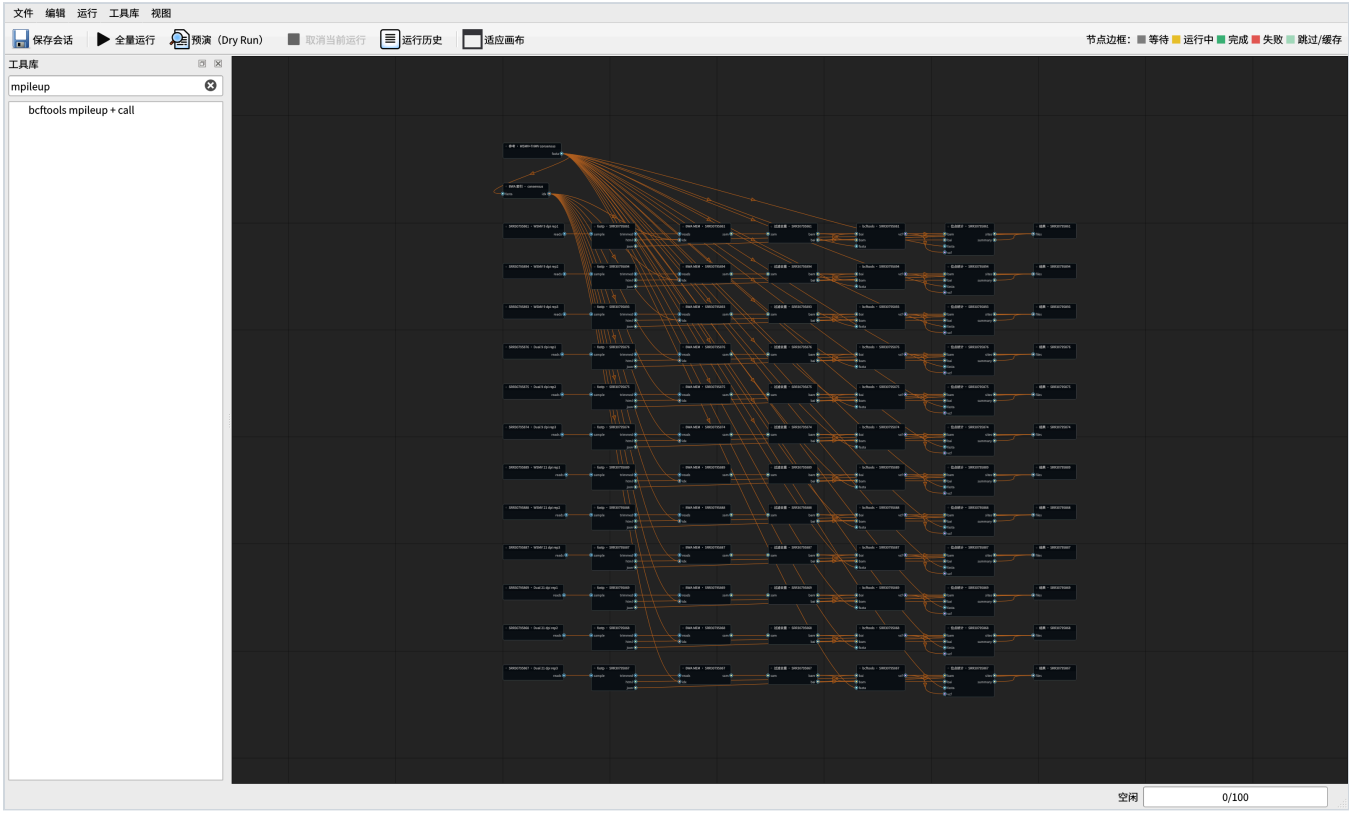
操作：启动 OpenBioWorkstation 后，在左侧工具库搜索 in:BAM，只显示以 BAM 为输入的节点。



看点：工具库按端口类型过滤；右侧属性面板在无选择时保持空白，底部日志保留界面初始化消息，画布提示从工具库拖入节点。

步骤 2　自动封装工具出现在工具库

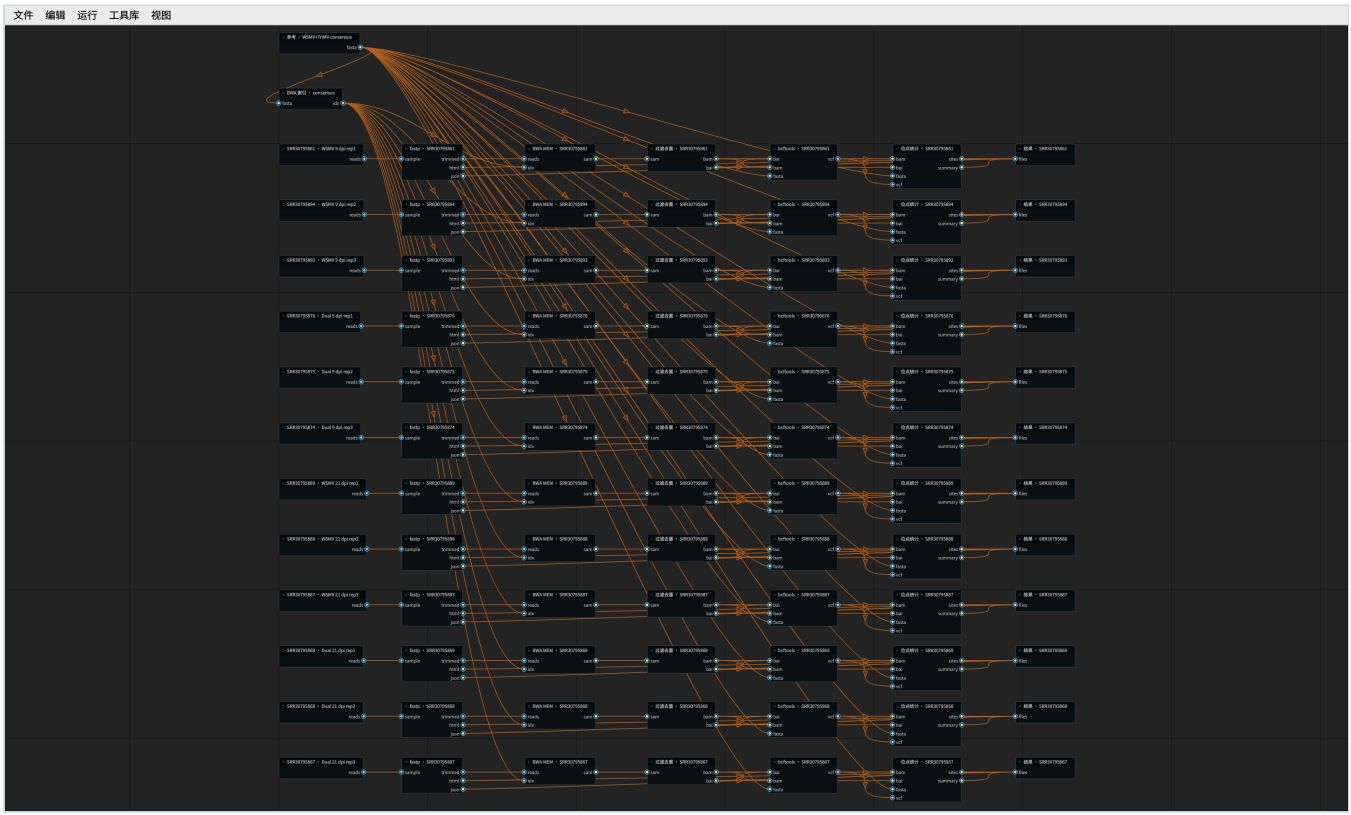
操作：在工具库搜索 mpileup，可见经 MCP auto_wrap_bioconda_pkg 封装的 bcftools mpileup 节点。



看点：自动封装节点与内置节点使用同一套端口/参数契约。属性中的 reviewed 标记由阶段 B 代理按任务授权设置，不是人工审核；定义已复制到 refs/，见文末说明（画布见步骤 3，节点文字见步骤 4）。

步骤 3　完整画布

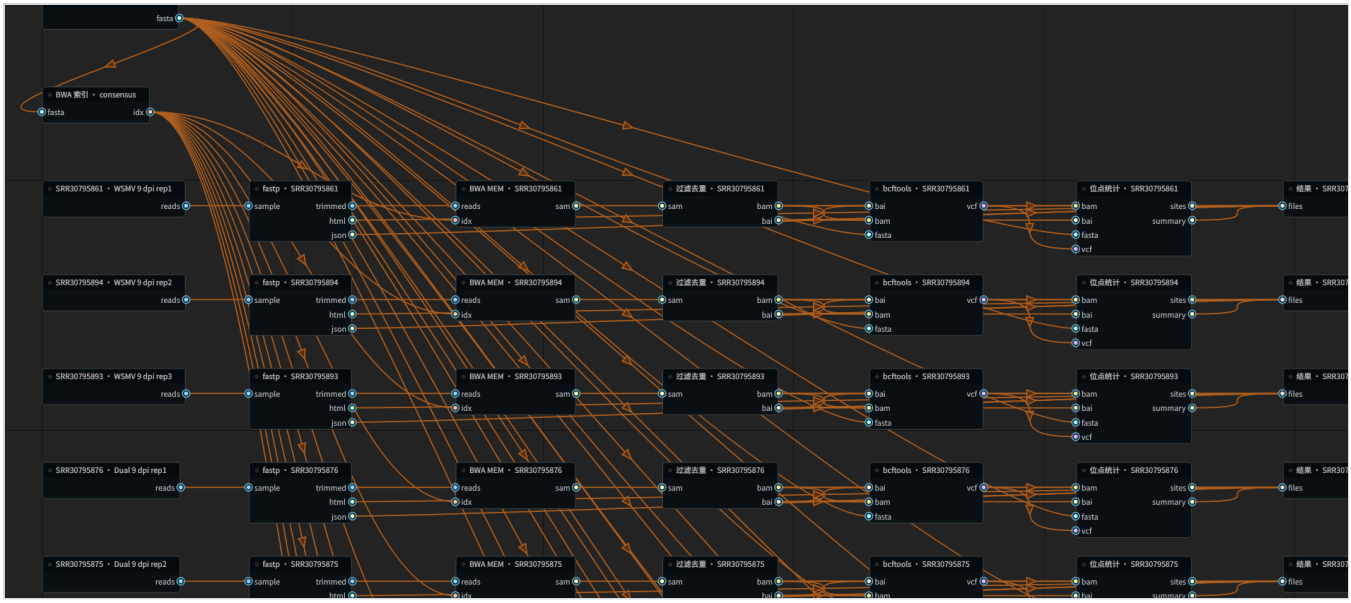
操作：通过“视图”菜单的“适应画布”，显示 12 条样本链、共享参考与 BWA 索引节点。



看点：86 个节点、205 条连线；每条链为 FASTQ → fastp → BWA MEM → 过滤去重 → bcftools → 位点统计 → 输出。截图为画布区域。

步骤 4　完整画布的局部放大（同图裁切）

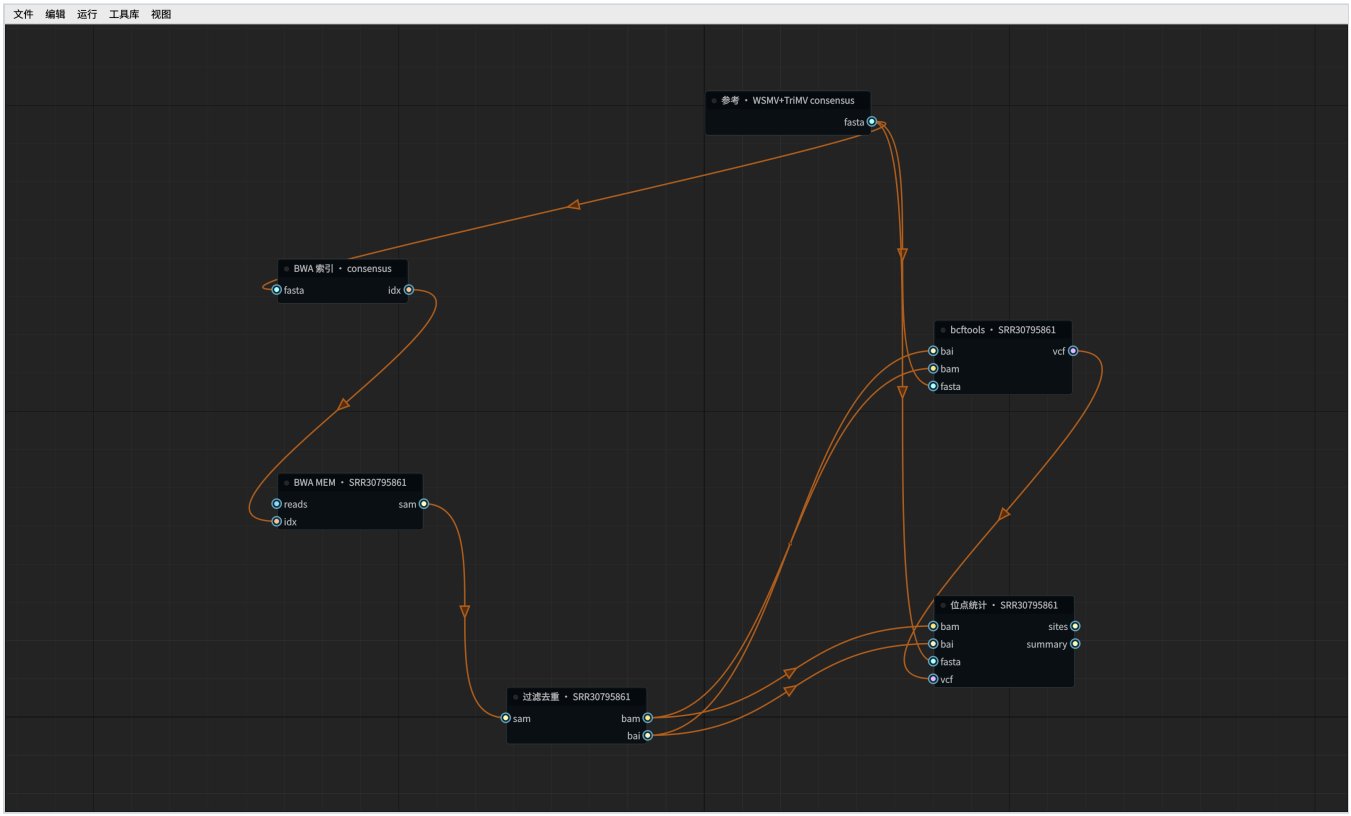
操作：对步骤 3 的截图裁取参考、索引与上方 4 条样本链，保留 2× 原始像素，不做缩放。



看点：局部放大后节点标题、端口名与连线走向可读；完整拓扑见步骤 3，共享节点接线见步骤 5。该图是同一张真实截图的裁切，不是重新绘制。

步骤 5　共享节点的接线细节

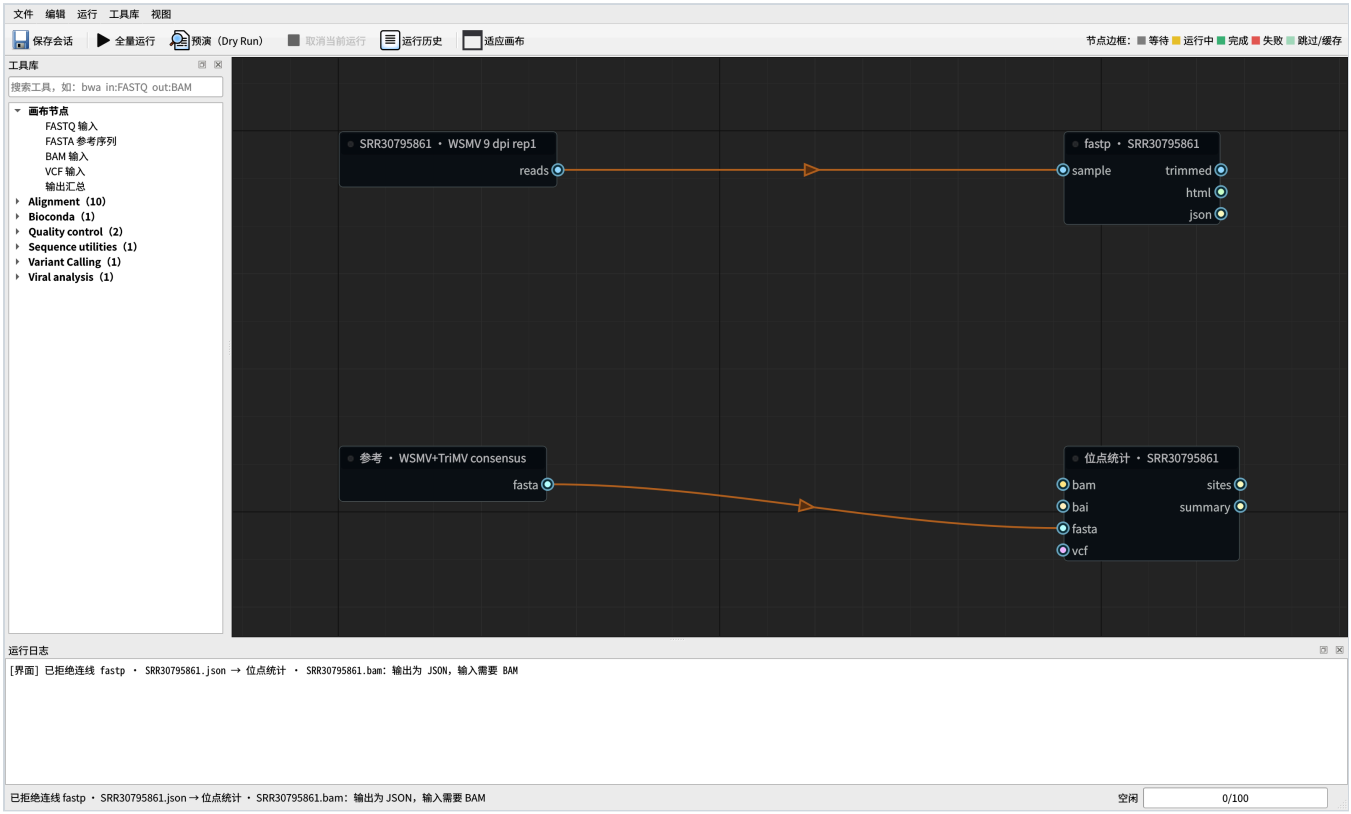
操作：该补充视图共保留 6 个节点：参考、索引，以及第 1 行样本链的 BWA MEM、过滤去重、bcftools 与位点统计；隐藏其余行与其连线，便于查看扇出起点。



看点：参考 fasta 同时进入 BWA 索引、bcftools 与位点统计；索引 idx 只进入 BWA MEM；过滤去重的 bam/bai 成对进入下游。

步骤 6　端口类型拦截

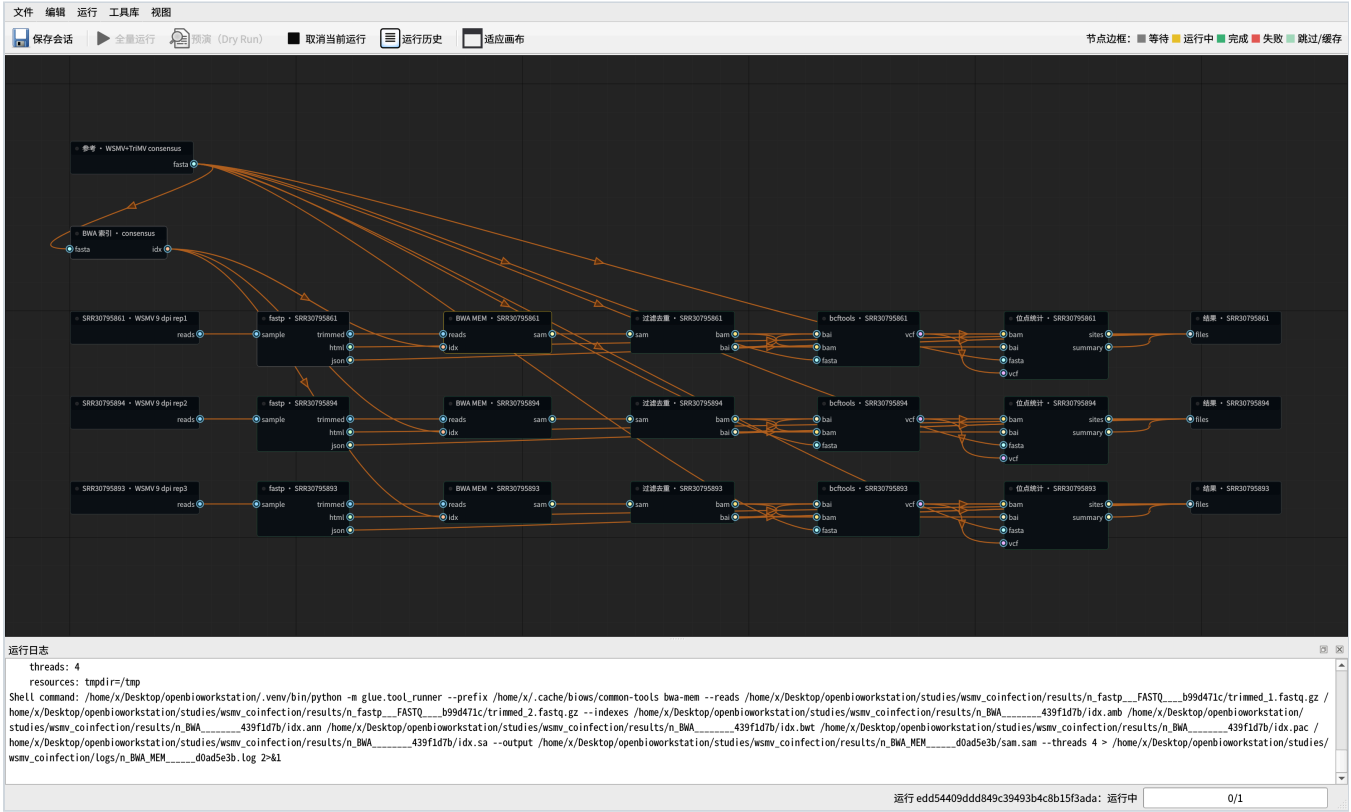
操作：尝试把 fastp 的 JSON 输出连接到“位点统计”的 BAM 输入。



看点：状态栏立即拒绝：“输出为 JSON，输入需要 BAM”。连线不会被创建，避免运行期才暴露格式错误。

步骤 7　运行中

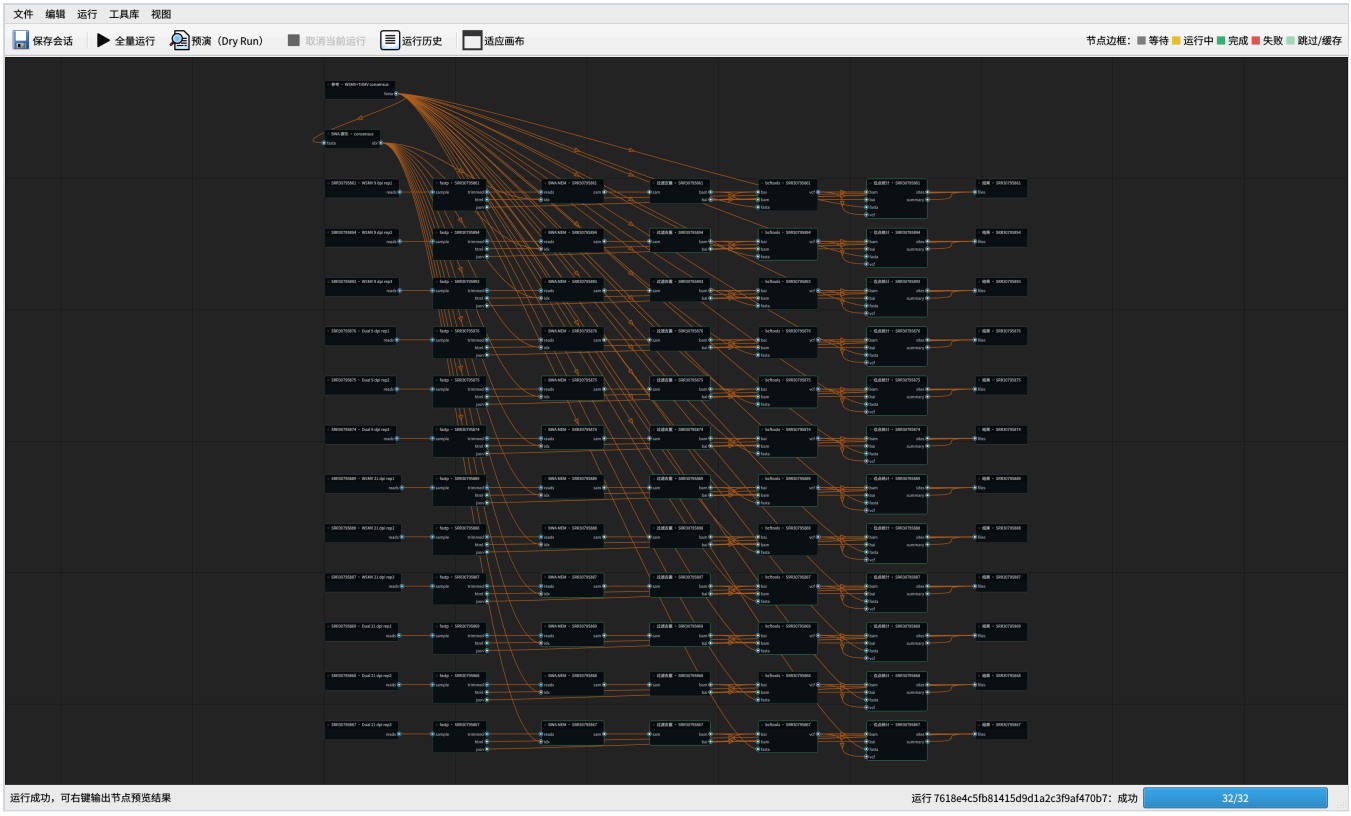
操作：以 --cores 4 启动运行；画布上正在执行与排队节点分别以黄色/灰色显示，底部进度条更新。



看点：Snakemake 按 DAG 调度；缓存命中与新增任务分开显示，便于判断进度。

步骤 8　运行完成

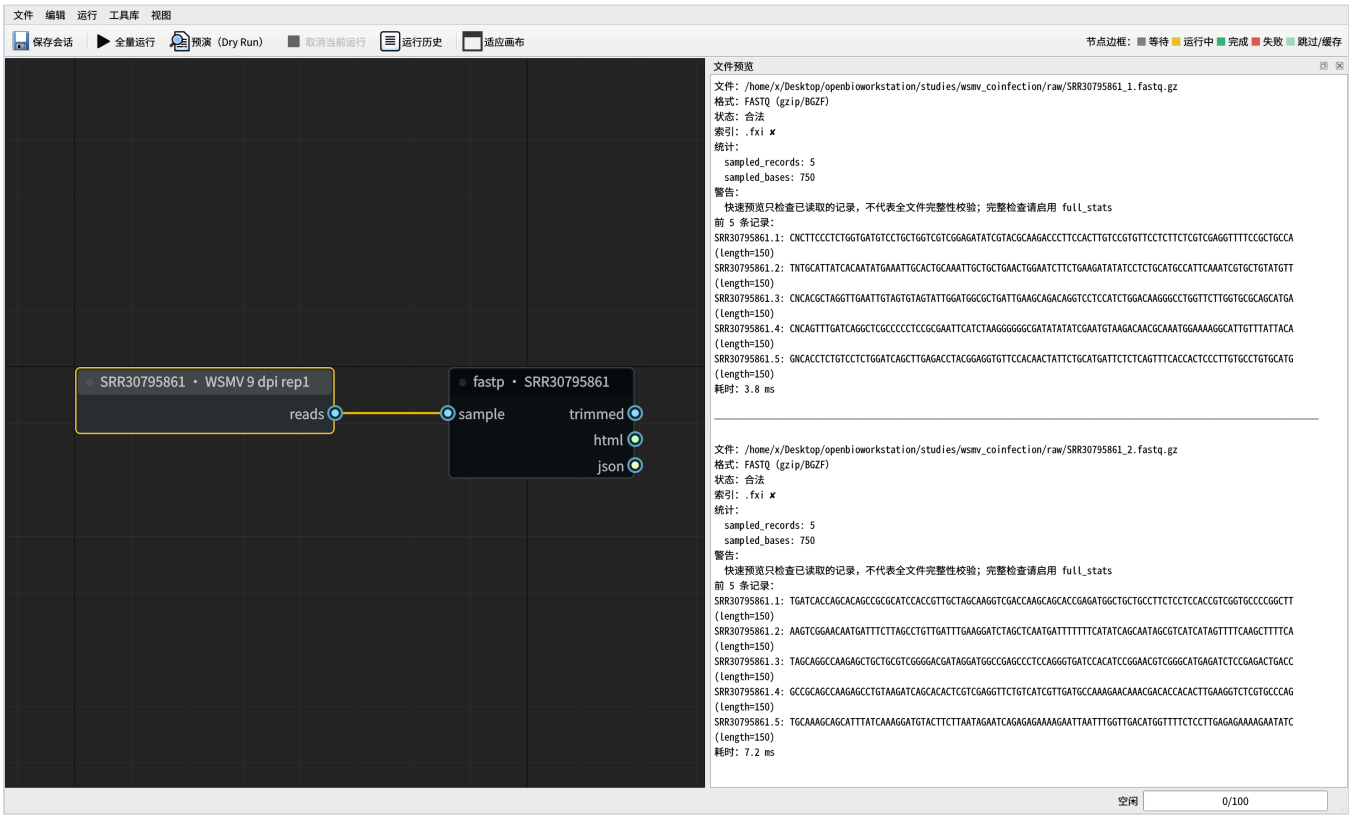
操作：已完成节点转绿、缓存命中节点显示浅绿，进度条到达 32/32。



看点：输出节点可直接用于预览；运行日志与每个规则的 benchmark 保存在 runs/ 与 benchmarks/。

步骤 9　输入 FASTQ 预览

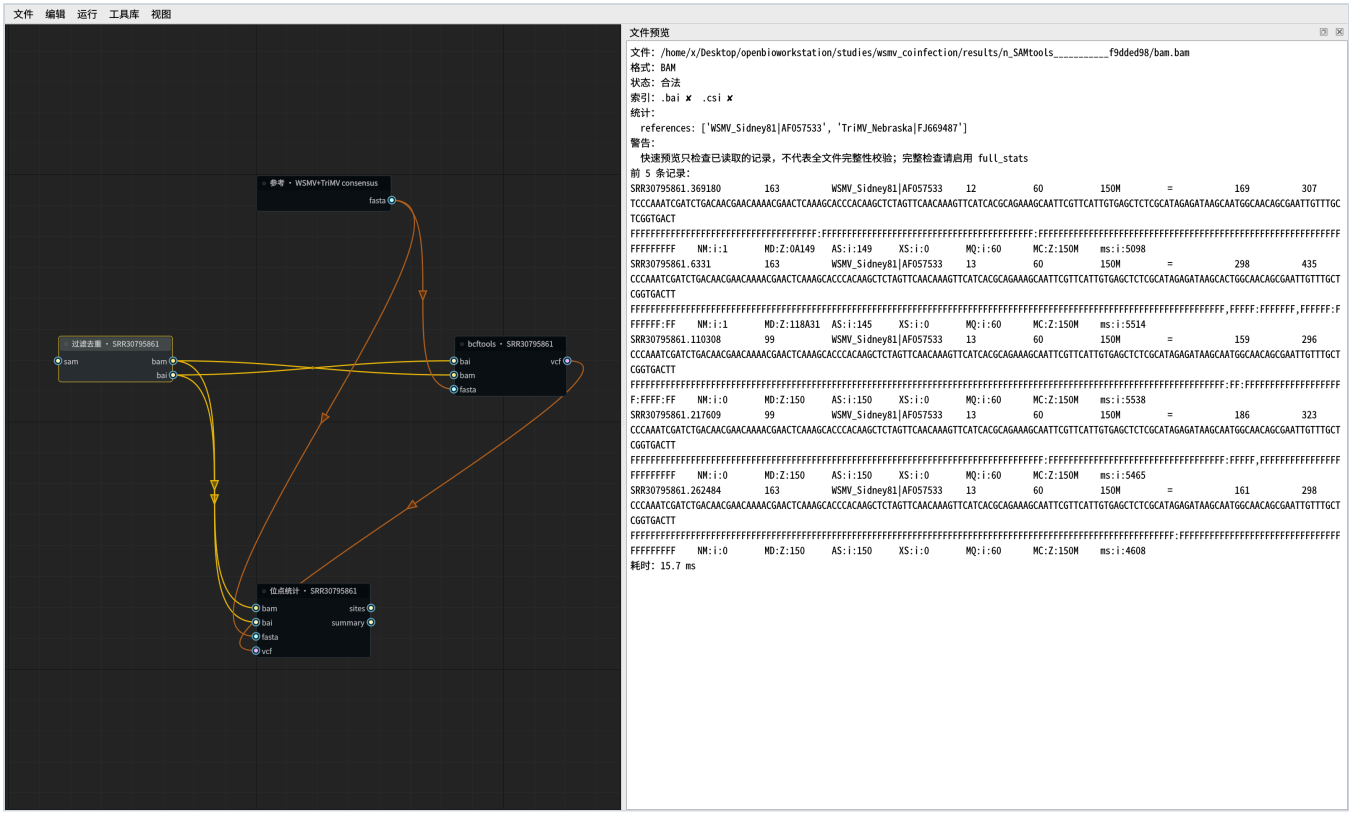
操作：选中输入节点，在预览面板查看 FASTQ 头部与质量信息。



看点：预览为只读；大文件按块读取，不阻塞界面。

步骤 10　过滤后 BAM 输出预览

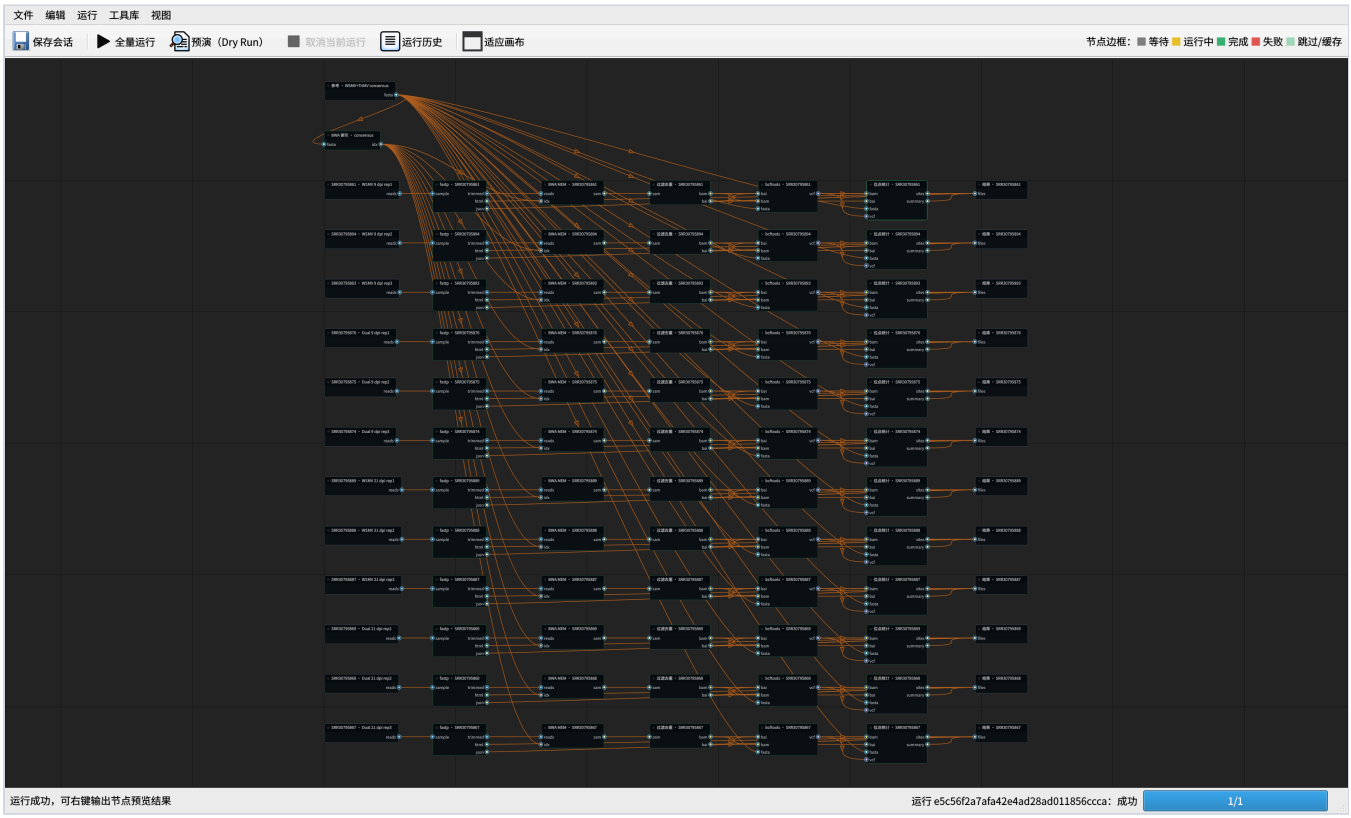
操作：选中“过滤去重”节点，预览其去重、排序后的 BAM 输出。



看点：每条记录含 read 名、flag、参考、位置、MAPQ、CIGAR 与序列；可结合 flag 与 CIGAR 核对去重与比对质量。

步骤 11　单步重跑与缓存

操作：把 SRR30795861 的 min_maf 从 0.02 改回主分析值 0.01，以 step 模式重跑位点统计节点。



看点：结果为 1 个节点执行、60 个节点 skipped：上游全部命中缓存，只重算目标节点，恢复主分析阈值。这是回退重跑；首次改为 0.02 时的 6/6 记录见步骤 12。

步骤 12　运行历史

操作：打开运行历史对话框，查看每次运行的开始时间、模式、目标节点、状态与进度。



开始时间	模式	目标节点	状态	进度	运行 ID
10-30 23:55:15	单步	BWA MEM · SRR30795861	成功	1/1	f2955782bd9f43588be3ddf124630ab2
10-30 23:54:28	单步	位点统计 · SRR30795861	成功	1/1	e5c56f2a7afa42e4ad28ad011856ccca
10-30 23:53:51	单步	位点统计 · SRR30795861	成功	6/6	ab29b25f340046e5a429751ea4b33930
10-30 21:42:57	单步	BWA MEM · SRR30795861	成功	1/1	8a93858a57e64edab1459f378a0d12d0
10-30 21:35:22	单步	BWA MEM · SRR30795861	成功	1/1	77ffe9f4f1534ecdadca85596a85d991
10-30 21:34:12	单步	BWA MEM · SRR30795861	成功	1/1	e3b4fcc246c94026a9e0cc09c08ef717
10-30 21:31:56	单步	BWA MEM · SRR30795861	成功	1/1	35f7d22d6a974a5e86c20f0f60e86df0
10-30 21:30:46	单步	BWA MEM · SRR30795861	成功	1/1	9264a1e940a24376aa72b10bb98cd442
10-30 21:25:55	单步	BWA MEM · SRR30795861	成功	1/1	3ada631c4f994bc6800e4ed80397d529
10-30 21:18:35	单步	BWA MEM · SRR30795861	成功	3/3	eb5db49289ea40d9b633fa535845766c
10-30 21:11:19	单步	BWA MEM · SRR30795861	成功	1/1	8a6a624574664cfbafd2db6863d9022a
10-30 20:43:00	单步	BWA MEM · SRR30795861	成功	1/1	27863c8f6272415a8f3b2862837ce134

Open　Close

看点：列表按开始时间倒序：首行为画布演示中启动的 BWA MEM 节点运行（1/1），其下为步骤 11 的回退重跑（目标节点，1/1），再往下是阈值 0.02 的首次重跑（6/6；BWA index → 位点统计共 6 个节点）。首次连上游也重算，是因为运行器代码在上一次全量运行后发生变化，Snakemake 一次性重算了该样本链的 6 个节点；回退时缓存已热身，所以只重算目标节点。

复现命令

A. 当前工作区重放（已有 raw/、ref/、work/pilot/）

```
cd /home/x/Desktop/openbioworkstation/studies/wsmv_coinfection

bash scripts/download.sh
.venv/bin/python scripts/make_consensus.py
../../.venv/bin/python scripts/workflow.py build --force
../../.venv/bin/python scripts/workflow.py run --cores 4
../../.venv/bin/python scripts/workflow.py rerun --sample SRR30795861 --param min_maf=0.02 --cores 4
../../.venv/bin/python scripts/workflow.py rerun --sample SRR30795861 --param min_maf=0.01 --cores 4
.venv/bin/python scripts/analyze_variants.py
.venv/bin/python scripts/make_figures.py
.venv/bin/python scripts/make_graphical_abstract.py
.venv/bin/python scripts/make_pdfs.py
```

B. 干净环境重建（从公开 reads 开始）

```
cd /home/x/Desktop/openbioworkstation

# B1. 工具环境与 Python 依赖
/opt/miniconda3/bin/conda env create -y -p ~/.cache/biows/common-tools   -f validation/common-tools-environment.yml
/opt/miniconda3/bin/conda create -y -p ~/.cache/biows/wheat-tools   -c conda-forge -c bioconda sra-tools=3.4.1
python3 -m venv .venv
.venv/bin/pip install -r requirements.txt
cd studies/wsmv_coinfection
python3 -m venv .venv
.venv/bin/pip install pysam==0.24.1 pandas==3.1.0 numpy==2.5.4   matplotlib==3.11.2 pypdf==6.21.4 reportlab==5.1.0
```

```
# B2. 参考：拼接两条 GenBank 序列并建索引
cat refs/WSMV_Sidney81.fa refs/TriMV_Nebraska.fa > refs/ws_tri_ref.fa
$HOME/.cache/biows/common-tools/bin/bwa index refs/ws_tri_ref.fa
$HOME/.cache/biows/common-tools/bin/samtools faidx refs/ws_tri_ref.fa

# B3. 试点下载与比对（4 个 run；WSMV 用 4 个、TriMV 用其中 2 个建共识）
SPOTS=200000 bash scripts/pilot_download.sh   SRR30795861 SRR30795876 SRR30795889 SRR30795869
bash scripts/pilot_align.sh SRR30795861 SRR30795876 SRR30795889 SRR30795869

# B4. 共识参考（含 TriMV 8233 T→A）
.venv/bin/python scripts/make_consensus.py

# B5. 正式下载与 auto-wrap 注册
bash scripts/download.sh
../../.venv/bin/python scripts/mcp_autowrap.py
# 可导入的封装定义与环境清单已复制到：
#   refs/auto_wrapped_bcftools_meta.yaml
#   refs/auto_wrapped_bcftools_environment.yaml

# B6. 画布、运行、统计、作图与 PDF
../../.venv/bin/python scripts/workflow.py build --force
../../.venv/bin/python scripts/workflow.py run --cores 4
.venv/bin/python scripts/analyze_variants.py
.venv/bin/python scripts/make_figures.py
.venv/bin/python scripts/make_graphical_abstract.py
.venv/bin/python scripts/make_pdfs.py
```

说明：A 部分是在现有工作区重放，依赖已生成的 pilot BAM 与共识；B 部分从公开 reads 开始，包含试用 SRA 下载、参考拼接、共识构建与 auto-wrap 注册。A 的画布步骤使用仓库 .venv（含 MCP 与 PySide6），其余分析步骤使用课题 .venv。下载支持断点续跑；已完成的 FASTQ 校验 reads 数后跳过。

审核状态说明：项目简报要求新节点经人工安装和审核；阶段 B 任务文件则要求代理代用户调用 mark_reviewed()。本演示按阶段 B 的授权执行，因此外部 bcftools 定义的 reviewed: true 是代理授权记录，不是独立真人审核。本次 D 阶段未修改该标记，也不声称已完成真人审核；若验收坚持人工审核，需补充独立的人工审核记录。